Week 2: SQL Injection and XSS Lab Exercises

Hands-on Exploitation, Vulnerability Root-Cause Analysis, and Secure Coding Defenses

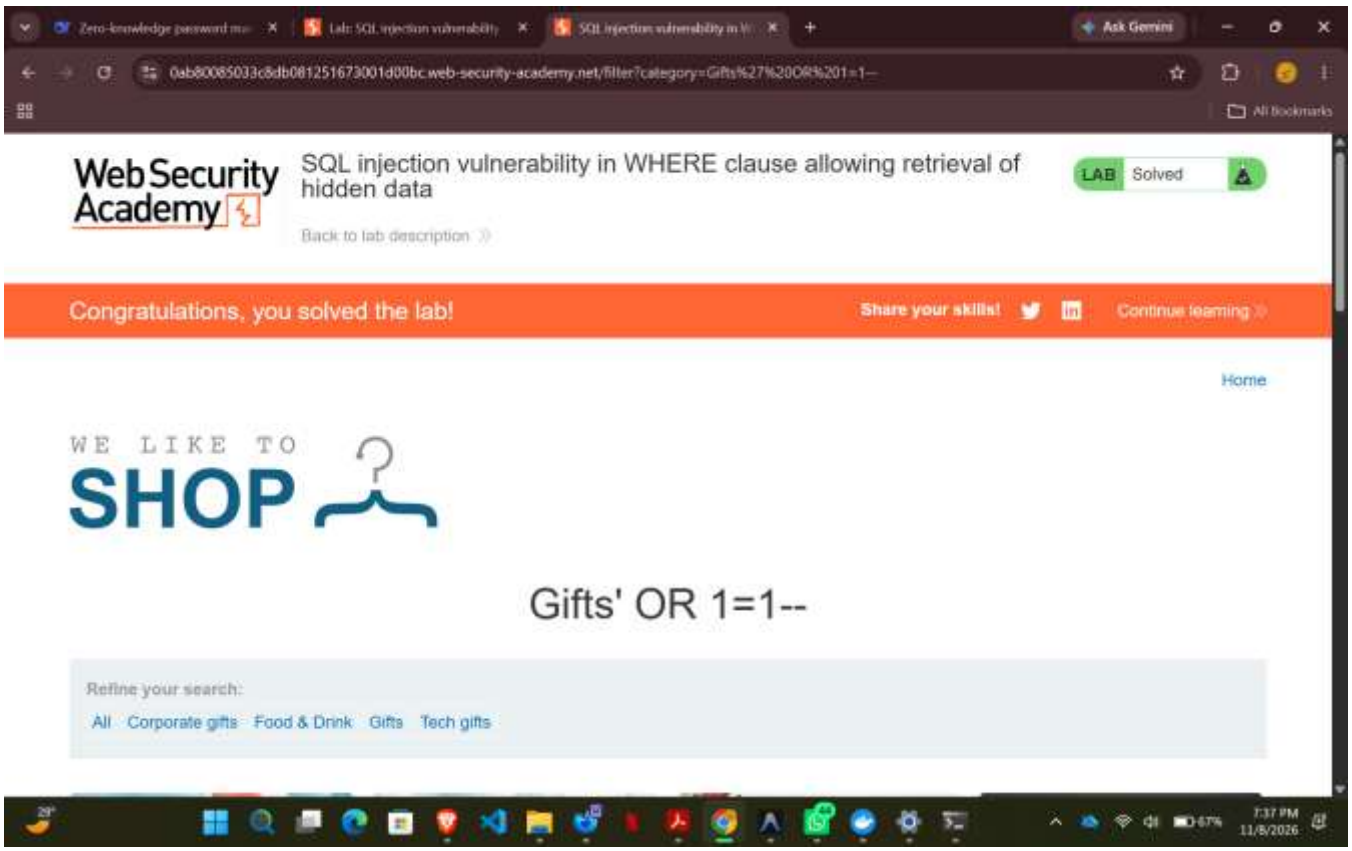
Team Roster (Group 6 BSc Software Engineering B)

• **Stedan Yaw Boah Ampofo** (ID: 1696596666)	• **Prince Lateef Abdul Rahman** (ID: 1703675677)
• **Keith Tetteh Yaotey** (ID: 1694883181)	• **Prince Opoku Owusu** (ID: 1702131132)
• **Sedoame Kofi Ahorlu** (ID: 1704464006)	• **Bright Kportiklah** (ID: 1693948139)
• **Cyril Prince Agbeve** (ID: 1701864653)	**Platform: PortSwigger Academy** | Status: Verified

1. Introduction

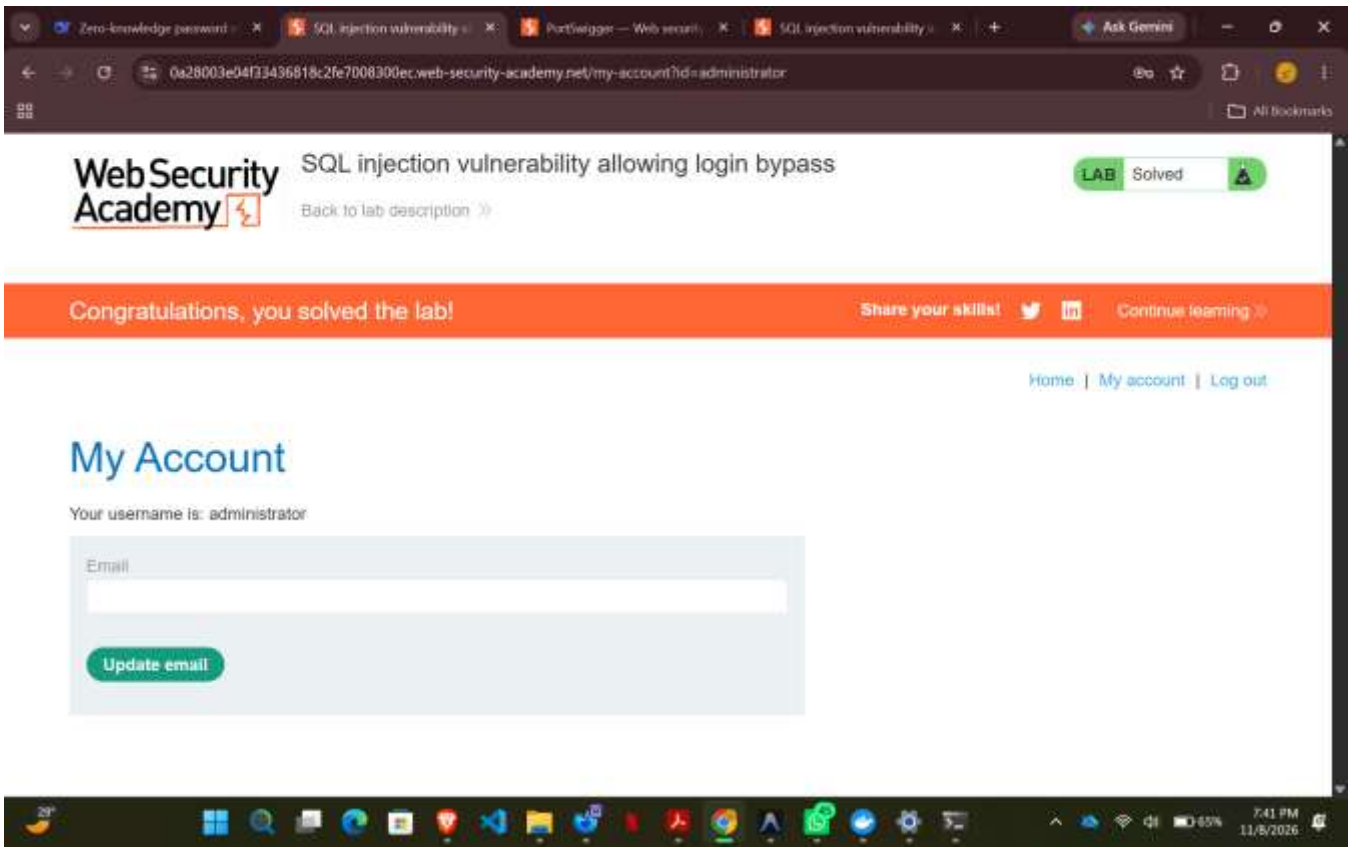
This report documents the completion of SQL Injection (SQLi) and Cross-Site Scripting (XSS) exercises on PortSwigger Web Security Academy. The objective was to exploit these vulnerabilities in a safe environment and identify the necessary defensive measures.

2. Lab 1: SQL Injection (WHERE Clause Hidden Data)



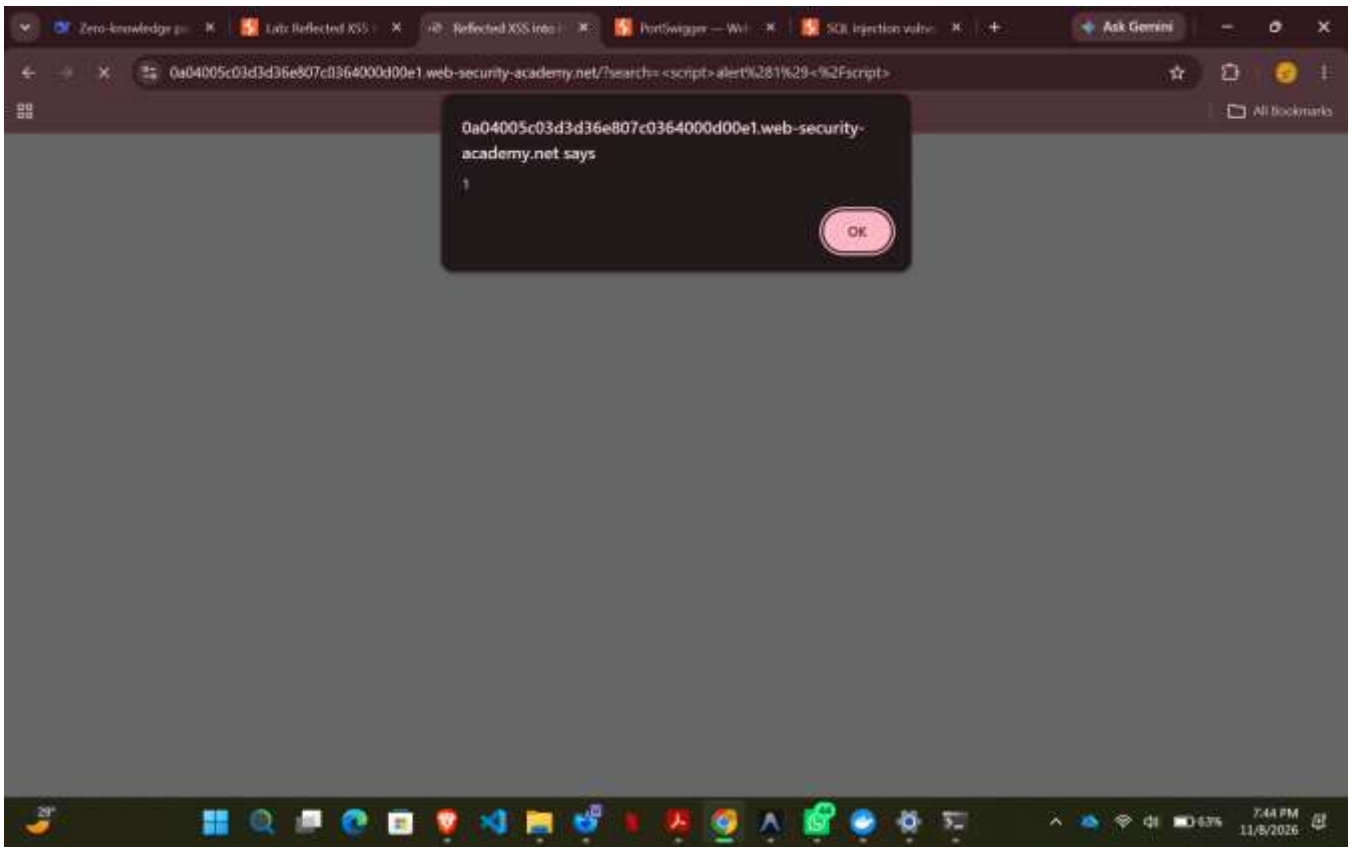
- **Payload Used: ' OR 1=1--**
- **Vulnerability Analysis:** User input was concatenated directly into the SQL query. The payload forced the query to evaluate as true for all rows and commented out the released = 1 check, returning hidden products.
- **Defensive Measure:** Use parameterized queries (prepared statements) to separate SQL code from user data.

3. Lab 2: SQL Injection (Login Bypass)



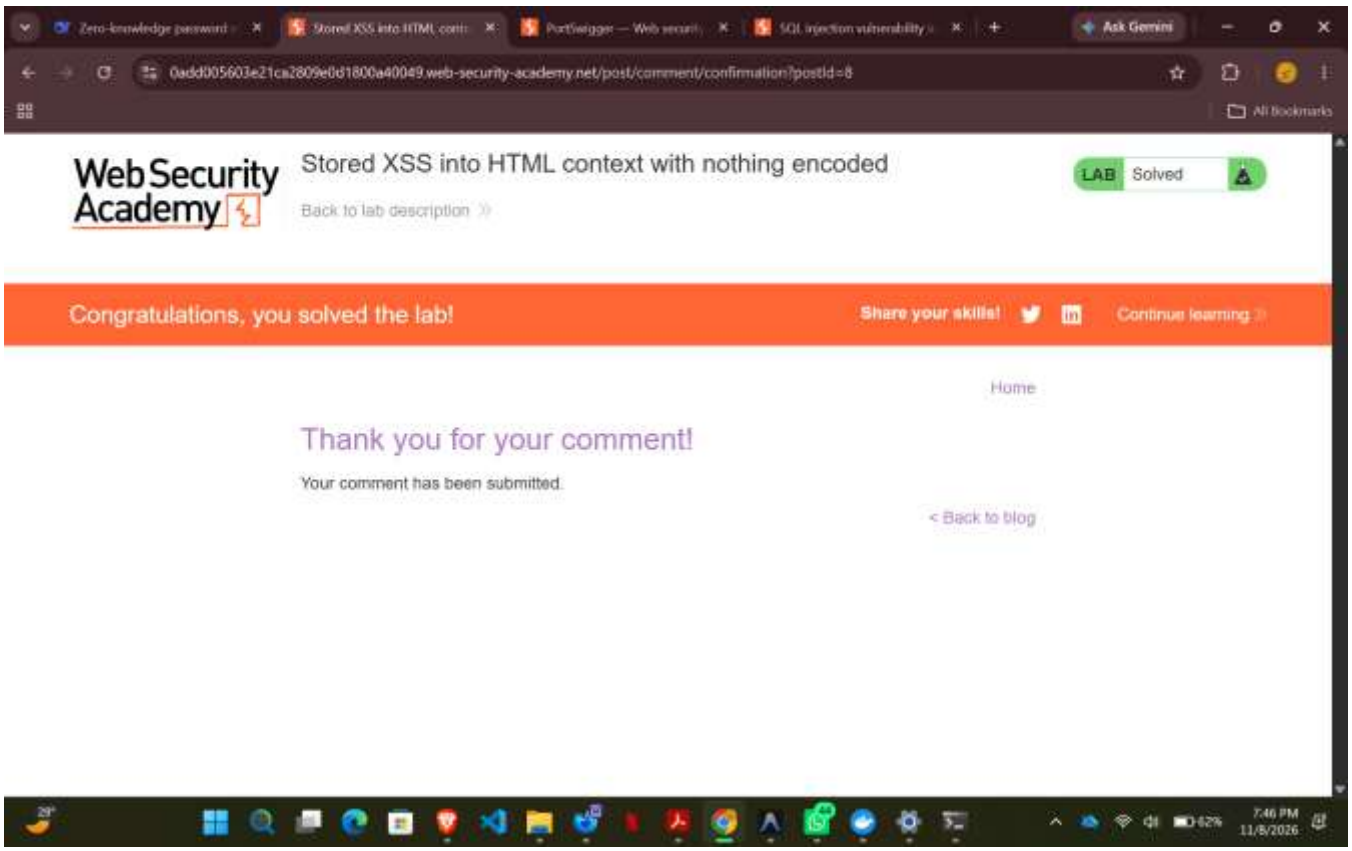
- **Payload Used: Username: administrator'--**
- **Vulnerability Analysis:** The payload commented out the password check in the login query, allowing unauthorized access as the admin user.
- **Defensive Measure:** Implement parameterized queries and generic error messages.

4. Lab 3: Reflected XSS



- **Payload Used: <script>alert(1)</script>**
- **Vulnerability Analysis:** User input from the search field was reflected into the HTML response without encoding, causing the browser to execute the script.
- **Defensive Measure:** Apply HTML entity encoding to all user-supplied data and implement a Content Security Policy (CSP).

5. Lab 4: Stored XSS



- **Payload Used: <script>alert(1)</script> in a blog comment.**
- **Vulnerability Analysis:** The comment was stored in the database without sanitization and executed whenever the page loaded.
- **Defensive Measure:** Sanitize input on the server side before storage, and encode output when rendering.

6. Conclusion

These exercises demonstrated the critical importance of input validation and output encoding. Secure coding practices like parameterized queries and HTML encoding are mandatory to protect against SQLi and XSS.